

Table des matières

Introduction.....	2
Partie 1 – Création des tables	3
Partie 3 – Création des contraintes	7
Partie 4.1 – Création des rôles utilisateur.....	8
Partie 4.2 – Création des logins et utilisateurs	9
Partie 4.3 – Création des vues	10
Partie 5 Requêtes interresantes	11
Partie 6 trigger	12
Partie 7 procédure	13
Conclusion	15

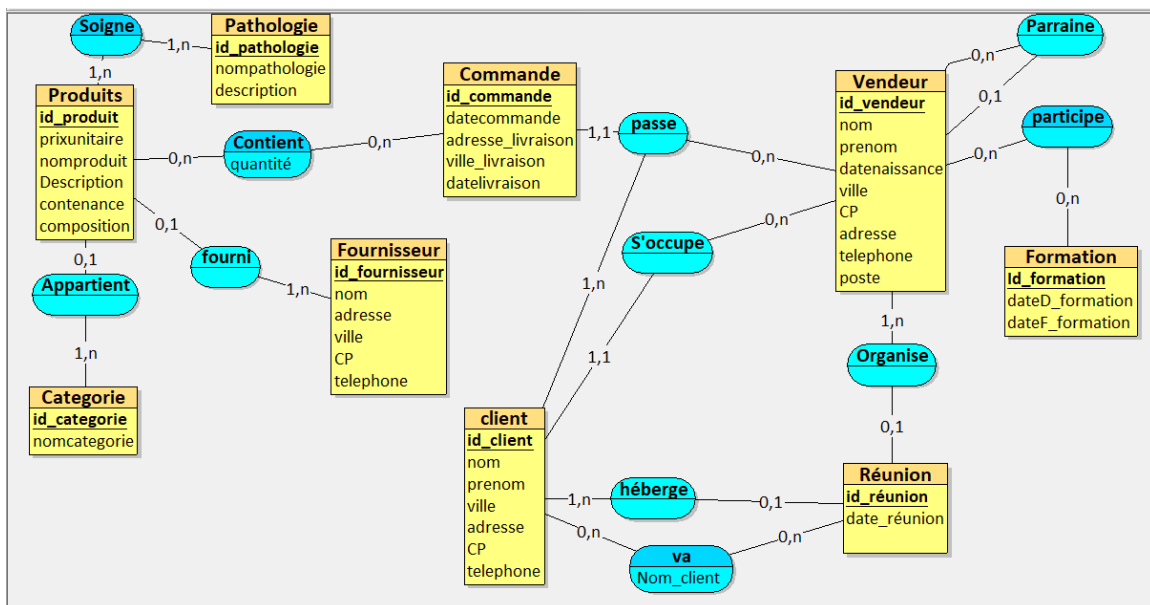
Introduction

Dans le cadre du projet de modélisation et de création d'une base de données pour la société Nature Derm, ce rapport présente l'ensemble du travail réalisé, depuis l'analyse initiale jusqu'à la mise en œuvre complète du script SQL. Conçu pour répondre aux besoins de gestion d'une filiale spécialisée dans la vente à domicile de cosmétiques paramédicaux, ce document expose de manière structurée les différentes étapes ayant permis d'aboutir à une solution fiable, cohérente et adaptée aux exigences du client.

Le rapport est organisé en sections thématiques afin de faciliter la compréhension du processus de conception. Chaque partie détaille les décisions prises, les modèles produits, ainsi que les choix techniques retenus pour la construction de la base. Les scripts SQL correspondants y sont présentés et accompagnés d'explications précises permettant de comprendre leur utilité, leur fonctionnement et leur rôle dans l'architecture globale du système.

Ce document constitue ainsi une vision complète et progressive du projet, depuis la réflexion conceptuelle avec la méthode Merise jusqu'à l'implémentation dans SQL Server, incluant la création des tables, l'ajout des contraintes, la gestion des utilisateurs, les requêtes, ainsi que les triggers et procédures nécessaires au bon fonctionnement de la base de données.

Présentation du Merise :



Partie 1 – Création des tables

Dans cette partie, nous avons créé plusieurs tables pour structurer les données d'une entreprise fictive, en lien avec les vendeurs, les clients, les produits, les commandes, les formations et les pathologies. Chaque table a un rôle précis dans la base de données.

Table : Vendeur

Cette table contient toutes les informations sur les vendeurs de l'entreprise : nom, prénom, date de naissance, adresse, téléphone, poste, etc.

La colonne `id_vendeur_superviseur` permet de relier un vendeur à son superviseur (autre vendeur), créant ainsi une hiérarchie entre les vendeurs.

Table : Client

La table Client enregistre les informations personnelles des clients (nom, prénom, adresse, téléphone).

Elle est liée à la table Vendeur via `id_vendeur` pour savoir quel vendeur s'occupe de chaque client.

Table : Pathologie

Cette table contient la liste des pathologies traitées par les produits de l'entreprise. Chaque pathologie a un nom et une description pour préciser de quoi il s'agit.

Table : Categorie

Cette table classe les produits par catégorie, ce qui permet d'organiser et de filtrer les produits plus facilement.

Exemple : médicaments, compléments alimentaires, matériel médical.

Table : Fournisseur

Cette table enregistre les informations sur les fournisseurs qui livrent les produits à l'entreprise : nom, adresse, téléphone, ville, etc.

Elle permet de savoir quel produit provient de quel fournisseur.

Table : Reunion

Cette table contient les informations sur les réunions organisées par les vendeurs avec les clients : date, client concerné, vendeur concerné.

Elle est reliée aux tables Client et Vendeur pour retrouver facilement les participants.

Table : Formation

La table Formation stocke les informations sur les formations suivies par les vendeurs : date de début et date de fin.

Elle permet de suivre les compétences et l'évolution des vendeurs.

Table : Commande

Cette table enregistre toutes les commandes passées par les clients : date, adresse de livraison, vendeur associé et client concerné.

Elle est reliée aux tables Vendeur et Client pour savoir qui a pris la commande et pour quel client.

Table : Produits

Cette table contient les informations sur les produits vendus : nom, description, composition, prix, contenance, fournisseur et catégorie.

Elle est reliée aux tables Fournisseur et Catégorie pour connaître la provenance et le type du produit.

Table : Contient

La table Contient est une table associative entre Commande et Produits.

Elle indique quels produits sont inclus dans chaque commande et en quelle quantité.

Table : Soigne

Cette table relie les produits aux pathologies qu'ils traitent.

Elle permet de savoir quel produit est adapté pour quelle pathologie.

Table : Participe

La table Participe relie les vendeurs aux formations auxquelles ils assistent.
Elle permet de suivre la participation des vendeurs aux différentes formations.

Table : Va

Cette table relie les clients aux réunions auxquelles ils participent.
Elle permet de retrouver rapidement quels clients étaient présents à quelle réunion.

Exemple de creation de table :*

Table vendeur :

```
CREATE TABLE Vendeur(  
  id_vendeur INT PRIMARY KEY,  
  nom VARCHAR(50),  
  prenom VARCHAR(50),  
  datenaissance DATE,  
  ville VARCHAR(50),  
  CP VARCHAR(50),  
  adresse VARCHAR(50),  
  telephone VARCHAR(10),  
  poste VARCHAR(50),  
  id_vendeur_superviseur INT NULL,  
  FOREIGN KEY(id_vendeur_superviseur) REFERENCES Vendeur(id_vendeur)  
);
```

Table Participe :

```
CREATE TABLE Participe(  
  id_vendeur INT,  
  id_formation INT,  
  PRIMARY KEY(id_vendeur, id_formation),  
  FOREIGN KEY(id_vendeur) REFERENCES Vendeur(id_vendeur),  
  FOREIGN KEY(id_formation) REFERENCES Formation(id_formation)  
);
```

Partie 2 – Insertion des données

Exemple d'insertions de valeurs :

Table vendeur :

INSERT INTO Vendeur VALUES

```
(1, 'Jean-Marie', 'Jung', '1980-05-15', 'Paris', '75010', '10 rue des Lilas', '0600000001',  
'Directeur', NULL),  
(2, 'Léana', 'Procida', '1992-09-21', 'Lyon', '69003', '25 rue Lumière', '0600000002',  
'Superviseur', 1),  
(3, 'Gordon', 'Chatton', '1998-12-02', 'Marseille', '13005', '14 boulevard Sud',  
'0600000003', 'Vendeur', 1),  
(4, 'Yannis', 'Elhajaoui', '1995-03-11', 'Toulouse', '31000', '55 rue Occitane',  
'0600000004', 'Vendeur', 2);
```

Table client :

```
(1, 'Bernard', 'Luc', 'Paris', '12 rue Foch', '75017', '0611223344', 3),  
(2, 'Petit', 'Anne', 'Lyon', '2 rue Rose', '69007', '0655667788', 4),  
(3, 'Durieux', 'Marc', 'Toulouse', '8 rue Soleil', '31000', '0677889900', 3),  
(4, 'Lambert', 'Julie', 'Marseille', '99 rue Mer', '13008', '0677001122', 4),  
(5, 'Moreau', 'Thomas', 'Nice', '22 rue des Orangers', '06000', '0612030405', 3),  
(6, 'Girard', 'Emma', 'Bordeaux', '15 rue du Lac', '33000', '0644556677', 4),  
(7, 'Faure', 'Hugo', 'Nantes', '7 allée Bleue', '44000', '0688995566', 3),  
(8, 'Garnier', 'Nina', 'Lille', '4 rue Lumière', '59000', '0677445566', 4),
```

Table Catégorie :

INSERT INTO Categorie VALUES

```
(1, 'Analgésiques'),  
(2, 'Dermatologie'),  
(3, 'Vitamines'),  
(4, 'Compléments'),  
(5, 'Antiseptiques'),  
(6, 'Digestifs'),  
(7, 'Cardio'),
```

Partie 3 – Création des contraintes

Dans cette partie, nous ajoutons des contraintes aux tables pour garantir l'intégrité des données. Les contraintes CHECK vérifient la validité des valeurs insérées ou mises à jour dans les colonnes.

Exemple de créations de contraintes :

-- Vérifie que la date de naissance d'un vendeur est dans le passé

```
ALTER TABLE Vendeur
```

```
ADD CONSTRAINT CK_Vendeur_DateNaissance CHECK (datenaissance <= GETDATE());
```

-- Vérifie que le numéro de téléphone du vendeur contient exactement 10 chiffres

```
ALTER TABLE Vendeur
```

```
ADD CONSTRAINT CK_Vendeur_Telephone CHECK (LEN(telephone) = 10 AND telephone NOT LIKE '%[^0-9]%');
```

-- Vérifie que le numéro de téléphone du client contient exactement 10 chiffres

```
ALTER TABLE Client
```

```
ADD CONSTRAINT CK_Client_Telephone CHECK (LEN(telephone) = 10 AND telephone NOT LIKE '%[^0-9]%');
```

-- Vérifie que le numéro de téléphone du fournisseur contient exactement 10 chiffres

```
ALTER TABLE Fournisseur
```

```
ADD CONSTRAINT CK_Fournisseur_Telephone CHECK (LEN(telephone) = 10 AND telephone NOT LIKE '%[^0-9]%');
```

-- Vérifie que le code postal du vendeur contient exactement 5 chiffres

```
ALTER TABLE Vendeur
```

```
ADD CONSTRAINT CK_Vendeur_CP CHECK (LEN(CP) = 5 AND CP NOT LIKE '%[^0-9]%');
```

-- Vérifie que le code postal du client contient exactement 5 chiffres

```
ALTER TABLE Client
```

```
ADD CONSTRAINT CK_Client_CP CHECK (LEN(CP) = 5 AND CP NOT LIKE '%[^0-9]%');
```

-- Vérifie que la date de livraison est soit nulle soit après la date de commande

```
ALTER TABLE Commande
```

```
ADD CONSTRAINT CK_Commande_DateLivraison CHECK (datelivraison IS NULL OR datelivraison >= datecommande);
```

Partie 4.1 – Création des rôles utilisateur

Création des rôles pour gérer les droits d'accès selon le profil (vendeur, superviseur, directeur).

Exemple de création de rôle :

```
CREATE ROLE R_Vendeur;
```

Crée un rôle R_Vendeur qui sera utilisé pour attribuer des permissions aux vendeurs.

```
GRANT SELECT, INSERT ON Commande TO R_Vendeur;  
GRANT SELECT ON Produits TO R_Vendeur;  
GRANT SELECT ON Client TO R_Vendeur;  
GRANT SELECT ON Contient TO R_Vendeur;  
GRANT SELECT ON Pathologie TO R_Vendeur;  
GRANT SELECT ON Commande TO R_Vendeur;  
GRANT SELECT ON Fournisseur TO R_Vendeur;  
GRANT SELECT ON Soigne TO R_Vendeur;  
Attribue au rôle R_Vendeur des droits de lecture (SELECT) et insertion (INSERT) sur  
certaines tables, pour permettre aux vendeurs de consulter et ajouter des commandes.
```

```
CREATE ROLE R_Superviseur;
```

Crée un rôle R_Superviseur pour les superviseurs avec plus de droits que les vendeurs.

```
GRANT SELECT, UPDATE ON Commande TO R_Superviseur;  
GRANT SELECT, UPDATE ON Produits TO R_Superviseur;  
GRANT SELECT, UPDATE ON Client TO R_Superviseur;  
GRANT SELECT, UPDATE ON Contient TO R_Superviseur;  
GRANT SELECT, UPDATE ON Pathologie TO R_Superviseur;  
GRANT SELECT, UPDATE ON Commande TO R_Superviseur;  
GRANT SELECT, UPDATE ON Fournisseur TO R_Superviseur;  
GRANT SELECT, UPDATE ON Soigne TO R_Superviseur;  
GRANT SELECT, Update ON Reunion TO R_Superviseur;  
GRANT SELECT, Update ON Vendeur TO R_Superviseur;  
GRANT SELECT ON Formation TO R_Superviseur;
```

Attribue au rôle R_Superviseur des droits de lecture et modification sur plusieurs tables pour gérer les données des vendeurs, clients, commandes et réunions.

```
CREATE ROLE R_Directeur;
```

```
GRANT CONTROL ON DATABASE::SN2_CBDD_Projet TO R_Directeur;
```

Crée le rôle R_Directeur avec contrôle total sur la base de données.

Partie 4.2 – Création des logins et utilisateurs

Création des logins pour se connecter à la base et association aux utilisateurs de la base.

Exemple de création de logins:

Crée un login L_JeanMarie avec un mot de passe, et définit la base par défaut et la langue.

(Les autres logins L_Leana, L_Gordon, L_Yannis, L_Directeur suivent le même principe.)

```
CREATE LOGIN L_JeanMarie
WITH PASSWORD = 'jean',
DEFAULT_DATABASE=SN2_CBDD_Projet,
DEFAULT_LANGUAGE=French,
CHECK_POLICY = OFF,
CHECK_EXPIRATION = OFF;
GO
```

```
DEFAULT_LANGUAGE=French,
CHECK_POLICY = OFF,
CHECK_EXPIRATION = OFF;
GO
```

```
CREATE LOGIN L_Directeur
WITH PASSWORD = 'admin',
DEFAULT_DATABASE=SN2_CBDD_Projet,
DEFAULT_LANGUAGE=French,
CHECK_POLICY = OFF,
```

```
CHECK_EXPIRATION = OFF;  
GO
```

```
CREATE USER U_JeanMarie FOR LOGIN L_JeanMarie;  
CREATE USER U_Directeur FOR LOGIN L_Directeur;
```

```
EXEC sp_addrolemember 'R_Superviseur', 'U_JeanMarie';  
EXEC sp_addrolemember 'R_Directeur', 'U_Directeur';
```

Partie 4.3 – Création des vues

Création de vues pour simplifier les requêtes et donner un accès filtré aux données selon le rôle.

Exemple de création de vues:

```
CREATE VIEW V_Ventes  
AS  
SELECT  
    co.id_commande,  
    c.nom AS client,  
    v.nom AS vendeur,  
    p.nomproduit,  
    ct.quantite,  
    p.prixunitaire,  
    ct.quantite * p.prixunitaire AS total_ligne  
FROM Commande co  
JOIN Client c ON co.id_client = c.id_client  
JOIN Vendeur v ON co.id_vendeur = v.id_vendeur  
JOIN Contient ct ON ct.id_commande = co.id_commande  
JOIN Produits p ON p.id_produit = ct.id_produit;
```

Vue V_Ventes qui montre les ventes avec les clients, vendeurs, produits, quantités et total par ligne.

```
GRANT SELECT ON V_Ventes TO R_Vendeur;
```

Donne le droit de lecture sur cette vue aux vendeurs.

```
CREATE VIEW V_Superviseur_Vendeurs
AS
SELECT
    v.id_vendeur,
    v.nom,
    v.prenom,
    v.poste,
    s.nom AS superviseur
FROM Vendeur v
LEFT JOIN Vendeur s ON v.id_vendeur_1 = s.id_vendeur;
```

Vue pour les superviseurs afin de voir les vendeurs et leur superviseur.

Partie 5 Requêtes interessantes

Top 5 des produits les plus vendus

```
SELECT TOP 5 p.nomproduit, SUM(c.quantite) AS totalVendu
FROM Contient c
JOIN Produits p ON c.id_produit = p.id_produit
GROUP BY p.nomproduit
ORDER BY totalVendu DESC;
```

Explication :

Cette requête calcule le **total des ventes pour chaque produit** en additionnant les quantités dans la table Contient. Elle affiche ensuite les **5 produits les plus vendus** en ordre décroissant.

Clients n'ayant passé aucune commande

```
SELECT c.nom, c.prenom
FROM Client c
LEFT JOIN Commande co ON co.id_client = c.id_client
WHERE co.id_client IS NULL;
```

Explication :

Cette requête utilise un **LEFT JOIN** pour lister tous les clients et vérifie ensuite lesquels **n'ont aucune commande** associée. Elle renvoie le nom et prénom de ces clients.

Liste des vendeurs avec leur superviseur

```
SELECT v1.nom AS Vendeur, v2.nom AS Superviseur  
FROM Vendeur v1  
JOIN Vendeur v2 ON v1.id_vendeur_superviseur = v2.id_vendeur;
```

Explication :

Cette requête relie la table Vendeur à elle-même pour obtenir **le nom du vendeur et le nom de son superviseur**.

Nombre de produits par fournisseur

```
SELECT f.nom AS fournisseur, COUNT(*) AS nbProduits  
FROM Fournisseur f  
JOIN Produits p ON f.id_fournisseur = p.id_fournisseur  
GROUP BY f.nom;
```

Explication :

Cette requête compte le **nombre de produits fournis par chaque fournisseur**, en groupant par le nom du fournisseur.

Commandes avec livraison retardée

```
SELECT id_commande, datecommande, datelivraison  
FROM Commande  
WHERE datelivraison > DATEADD(day, 3, datecommande);
```

Explication :

Cette requête affiche les commandes dont la **date de livraison dépasse de plus de 3 jours la date de commande**, permettant d'identifier les retards de livraison.

Partie 6 trigger

Un trigger, également appelé déclencheur, permet d'exécuter un ensemble d'instruction SQL juste après un événement. Cela permet de faciliter et d'automatiser des actions au sein d'un Système de Gestion de Base de Données

Exemple de triggers :

-- Modification tables

```
ALTER TABLE Client ADD nb_commandes INT DEFAULT 0;
```

-- Trigger qui permet d'augmenter de 1 le total de commande par client

```
CREATE TRIGGER TR_Client_NbCommandes
ON Commande
AFTER INSERT
AS
BEGIN
    UPDATE Client
    SET nb_commandes = nb_commandes + 1
    FROM Client c
    JOIN inserted i ON c.id_client = i.id_client;
END;
```

-- Verifie si la qt ajouter est bien supérieure a 0

```
CREATE TRIGGER TR_Verif_Quantite
ON Contient
AFTER INSERT
AS
BEGIN
    IF EXISTS(SELECT 1 FROM inserted WHERE quantite <= 0)
    BEGIN
        RAISERROR('La quantité doit être > 0',16,1);
        ROLLBACK TRANSACTION;
    END
END;
```

Partie 7 procédure

Une procédure stockée, aussi appelée stored procedure en anglais, est un concept utilisé en administration de base de données afin d'exécuter un ensemble d'instructions SQL. Une telle procédure est stockée au sein du Système de Gestion de Base de Données (SGBD) et peut être appelée à tout moment par son nom afin d'exécuter celle-ci.

Exemple de creations de procedures:

```
CREATE PROCEDURE PS_TotalVendeur
```

```
    @id_vendeur INT
```

```
AS
```

```
BEGIN
```

```
    SELECT SUM(c.quantite * p.prixunitaire) AS totalVentes
```

```
    FROM Commande co
```

```
    JOIN Contient c ON c.id_commande = co.id_commande
```

```
    JOIN Produits p ON p.id_produit = c.id_produit
```

```
    WHERE co.id_vendeur = @id_vendeur;
```

```
END;
```

```
CREATE PROCEDURE PS_RechercheProduit
```

```
    @motcle VARCHAR(50)
```

```
AS
```

```
BEGIN
```

```
    SELECT *
```

```
    FROM Produits
```

```
    WHERE nomproduit LIKE '%' + @motcle + '%'
```

```
    OR description LIKE '%' + @motcle + '%';
```

```
END;
```

Conclusion

Ce projet nous a permis de concevoir et de mettre en œuvre une base de données complète pour la société Nature Derm, depuis la modélisation conceptuelle avec la méthode Merise jusqu'à l'implémentation dans SQL Server.

À travers les différentes étapes du rapport, nous avons structuré les données autour de tables cohérentes (vendeurs, clients, produits, commandes, formations, etc.), tout en garantissant l'intégrité des données grâce à des contraintes CHECK et des clés étrangères. La gestion des accès a été assurée par la création de rôles distincts (vendeur, superviseur, directeur) associés à des utilisateurs et des vues adaptées à chaque profil, ce qui permet une sécurité fine et une séparation claire des responsabilités.

Les requêtes présentées illustrent la capacité de la base à répondre à des besoins métier concrets : identification des produits les plus vendus, suivi des clients inactifs, hiérarchie des vendeurs ou encore détection des retards de livraison. Les triggers et procédures stockées ajoutent une couche d'automatisation qui améliore la fiabilité et la maintenabilité du système.

En définitive, cette base de données constitue une solution robuste et évolutive, adaptée aux besoins opérationnels de Nature Derm. Elle pourrait être enrichie à l'avenir par des fonctionnalités supplémentaires telles que des tableaux de bord analytiques, une gestion des stocks en temps réel, ou encore l'intégration d'un système de fidélisation client.